This runs the command on the instance by logging in with SSH. Typically, this is more convenient for one-off commands to avoid having to SSH in and terminate a session manually.

Let's try this now:

```
$ docker-machine ssh local-vm-1 "echo 'hello'"
hello
```

As you can see, this ran the command on our local VirtualBox instance and displayed the output.

Configuring the Docker CLI

There's yet another way to perform actions on our docker-machine-managed instance: we can configure our (local) Docker client to talk to our new instance's Docker Engine. This is a huge benefit of Docker's client-server architecture we saw earlier on page 13. We can use the same commands with the client CLI locally and have it take effect on any remote machine of our choosing; we simply configure our CLI to point to different machines.

Configuring the Docker CLI to point at a different instance's Docker Engine is done via environment variables. However, we don't have to manage them manually; Docker Machine provides a shortcut. If you look back at the output when we created our new instance, the final line said:

```
To see how to connect your Docker Client to the Docker Engine running on
this virtual machine, run: docker-machine env local-vm-1
```

Let's run this now:

```
$ docker-machine env local-vm-1
export DOCKER_TLS_VERIFY="1"
export DOCKER_HOST="tcp://192.168.99.100:2376"
export DOCKER_CERT_PATH="/Users/rob/.docker/machine/machines/local-vm-1"
export DOCKER_MACHINE_NAME="local-vm-1"
# Run this command to configure your shell:
# eval $(docker-machine env local-vm-1)
```

This command prints the environment variables that must be set, but doesn't actually set them. To set them, we can follow the instructions in the output and run:

```
$ eval $(docker-machine env local-vm-1)
```

Our current terminal session is now configured so our Docker commands will run on our new, virtual instance, rather than our normal Docker installation.